SYSTEM
PID=1
human

POLYMATH OS

LFS-style rebuild of the personal operating system.
Best-in-class практики 2024-2026.

> Человек = система = processor + сумма частей. Репозиторий как self-bootstrapping операционная единица индивида-полимата: repo structure, AI workflows, second brain, devex/CI в одном дереве.

DOCUMENT TYPE
Technical Report

AUDITED DOMAINS
4 + cross-cutting

BUILD MODE
LFS — from scratch

DATE
2026-10-16

Содержание

Резюме 3

Философская основа 4

Тезис 1. Целое = Человек = Процессор . . . 4

Тезис 2. parole > langue (по Соссюру) . . . 4

Тезис 3. LFS — этическая позиция . . . 4

Тезис 4. PID=1 общий, security context отдельный . . . 4

Методология 5

Layer 1: Repository Structure 6

Принципы . . . 6

Структура (канон) . . . 6

Стек . . . 6

Анти-паттерны . . . 7

LFS-рекомендации . . . 7

Layer 2: AI Workflows 9

Принципы . . . 9

Стек . . . 9

Ключевые паттерны . . . 10

Анти-паттерны . . . 10

Layer 3: Second Brain / PKM 12

Принципы . . . 12

Стек (LFS-rebuild defaults) . . . 12

Структура vault . . . 12

Анти-паттерны . . . 13

Layer 4: DevEx / CI / IaC / Dotfiles 14
Принципы 14
Стек (defaults) 14
Ключевые паттерны 15
Синтез: Архитектура Polymath OS 16
Эмерджентные свойства 16
Слои и cross-cutting 16
Поток данных 16
Техническое задание (сводка) 17
Цель 17
Критерии приёмки 17
Non-goals 17
Дорожная карта (сводка) 18
Что после v1.0.0 18
Заключение и следующие шаги 19
Что делать сейчас 19

Резюме

Этот отчёт описывает **LFS-style rebuild** личной операционной системы индивида-полимата: единый self-bootstrapping репозиторий, объединяющий четыре домена — структуру монорепо, AI-рабочие процессы, личную базу знаний и DevEx/CI/IaC. Подход вырос из философии «человек = система = processor + сумма частей»: репозиторий рассматривается как материальная проекция init-процесса (PID=1), в которой каждый инструмент — fork-exec от человека, а не внешний зависимость.

Аудит покрыл 4 домена с фокусом на лучшие мировые практики 2024-2026: trunk-based development, MCP-нативный AI-стек, local-first PKM на markdown, OpenTofu + Nix + chezmoi для воспроизводимой среды. Для каждого домена выделены принципы, инструменты, паттерны, анти-паттерны и конкретные шаги LFS-style rebuild. Итоговая архитектура эмерджентна: репозиторий одновременно является базой кода, second brain, AI-стеком и dev-средой, с self-audit и self-publish свойствами, которых нет ни в одном слое отдельно.

> *Цель: v1.0.0 production-ready за 90 дней, 4 фазы по ~22 дня. Каждая фаза — слой бутстрапа, без которого следующая невозможна. От git init на пустом каталоге до self-hosted observable deployable system с SBOM, cosign signatures и SLSA Level 2+ provenance.*

Философская основа

Polymath OS вырос не из технического стека, а из ряда философских тезисов, выведенных в диалоге и проверенных реальностью. Эти тезисы формируют «конституцию» системы: каждое архитектурное решение должно быть совместимо с ними, иначе оно технический долг.

Тезис 1. Целое = Человек = Процессор

Человек — это не один файл в системе. Человек — это **PID=1**, init-процесс. Все остальные «файлы» (роли, проекты, навыки, базы знаний) — fork-exec от него: тот же код, тот же исток, разные образы. Это техническая модель того, что в традиции называется «лицо» (persona) — первичная неделимая единица, из которой нарезаны все социальные роли и институции.

Тезис 2. parole > langue (по Соссюру)

Индивидуальное слово (*parole*) первично. Коллективный код (*langue* — договор, консенсус, корпорация, государство) — производная, всегда меньше человека. Поэтому любая институция вторична перед лицом человека. Это объясняет, почему vendor lock-in и opinionated frameworks против твоей воли — форма институционального захвата: попытка langue подменить parole.

Тезис 3. LFS — этическая позиция

Linux From Scratch — не технический выбор, а этический. Никаких предсобранных бинарников. Никакой внешней зависимости от чужих корпоративных сборок. Бутстрап из source, из истока. Человек бутстрапит себя сам — это применяется и к репозиторию: каждый компонент должен быть понятен, заменим, откатываем. Если ты не можешь объяснить себе, что делает та или иная зависимость — это уже не твой код.

Тезис 4. PID=1 общий, security context отдельный

`fork()` копирует образ процесса, но **не копирует креды**. Я и Ты — один исток, но разные сессии. Это техническая основа индивидуальной автономии: даже если философски мы — один init, на уровне доступа у каждого своя сессия со своими правами. Поэтому делегирование «карт-бланша» не делает второго тебя собой на уровне безопасности — это session isolation, не отсутствие идентичности.

Методология

Аудит выполнен через 4 параллельных исследовательских агента, каждый из которых провёл web-research в своём домене и вернул структурированный markdown-отчёт (2500-3500 слов каждый). Источники: форумы (Reddit r/ObsidianMD, r/PKMS, r/selfhosted), блоги практикующих (zettelkasten.de, notes.andymatuschak.org, fortelabs.com, linkingyourthinking.com), GitHub-репозитории плагинов и инструментов, официальная документация (Anthropic engineering blog, Google developers blog, OpenTelemetry, Sigstore, SLSA, CycloneDX), и обзоры 2025-2026.

Каждый отчёт следовал единой структуре: Принципы (the WHY), Инструменты (the WHAT), Паттерны (the HOW), Анти-паттерны (что избегать), LFS-рекомендации (конкретные шаги rebuild с нуля), Референсы (URLs best-in-class публичных репозиториев). После сбора данных был выполнен синтез: 4 домена объединены в единую архитектуру Polymath OS с явными ADR-связями между принципами и решениями.

Домен	Фокус	Длина отчёта	Ключевой вывод
Repository structure	Monorepo, trunk-based, templates	~3000 слов	Repo-as-everything, linear history, CODEOWNERS
[illegible]	[illegible]	[illegible]	[illegible]
Second brain / PKM	Obsidian, git, atomic notes	~3200 слов	Markdown-first, local-first, output > collection
[illegible]	[illegible]	[illegible]	[illegible]

Layer 1: Repository Structure

Репозиторий — это не только код. Это инфраструктура (OpenTofu), документация (markdown как first-class citizen), конфигурация CI/CD, политики безопасности, операционные рунбуки. Всё, что нужно для воспроизведения системы на чистой машине, должно лежать в git. Это устраняет «works on my machine» и исчезнувшего коллегу как single point of failure.

Принципы

- **Repository as single source of truth.** Repo-as-everything: код + IaC + docs + политики.
- **Trunk-based development.** Короткоживущие ветки (≤24 ч), частая интеграция в main. GitFlow — только для boxed on-prem.
- **Linear history + squash-merge.** Один атомарный коммит на единицу работы. Conventional Commits как lingua franca.
- **Code ownership.** Каждый path в CODEOWNERS. PR не смержится без approval владельцев затронутых файлов.
- **Documentation-driven design.** README-first, ARCHITECTURE.md, ADR, RUNBOOK — четыре слоя живой документации.
- **Supply chain integrity.** SBOM (CycloneDX), SLSA Level 2+, cosign keyless signing.
- **Automation over discipline.** Правило в коде = blocking. Discipline processes деградируют за квартал.

Структура (канон)

Turborepo-стиль канон, подтверждённый практикой Vercel, T3-oss и публичными template-repos:

```
apps/ — деплоящиеся приложения (имеют next build и Dockerfile).
packages/ — переиспользуемые пакеты (имеют package.json с @scope/name).
tooling/ — generators, codemods.
docs/ — first-class документация, включает ADR.
.changeset/, .github/workflows/, .devcontainer/, turbo.json, pnpm-workspace.yaml —
meta-infra.
```

Стек

Инструмент	Назначение	Когда использовать
pnpm workspaces	Менеджер монорепо, symlink'и вместо hoisting	≤3 пакетов, маленькие JS/TS монорепо
Turborepo	Build system с content-aware hashing + remote cache	3–50 пакетов, нужен быстрый CI
Nx	Полноценный monorepo с generators	Enterprise (50+ пакетов)
Changesets	Управление версиями + changelog + publish	JS/TS монорепо, опционально
Conventional Commits	Спецификация коммитов (feat:, fix:, BREAKING:)	Везде — lingua franca
GitHub Merge Queue	Serialization PR для green trunk	Все контрибьюторы
Graphite / PRPlatter	Stacked PRs	Длинные фичи в trunk-based
Cosign	Keyless signing артефактов	Любой публичный релиз
Sigstore / Cosign	Keyless signing артефактов	Любой публичный релиз

Анти-паттерны

- GitFlow в новом продукте (зарезервированные develop, release/*, hotfix/*)
- Long-lived feature branches (>3 дней) — генерируют конфликты
- Вручную поддерживаемый CHANGELOG.md (забывается в 80% PR'ов)
- Документация в Confluence/Notion (умирает за 6 месяцев)
- README = installation guide на 300 строк (должен быть landing page)
- Нет CODEOWNERS — review у случайных людей
- Merge commits без squash — шум в git log
- Прямой push в main — даже для админов
- Lerna в новом монорепо (deprecated в пользу Turborepo + Changesets)
- yarn.lock + package-lock.json в одном репо
- .env закоммичен (даже с fake values)
- Нет SBOM в release pipeline

LFS-рекомендации

LFS-style rebuild стартует с пустого каталога, без легаси и без предсобранных бинарников. Шаги (детально в ROADMAP.md, Фаза 0):

- **Шаг 1.** `git init -b main` + структура каталогов apps/packages/tooling/docs.
- **Шаг 2.** Базовые метафайлы: README.md, LICENSE, CONTRIBUTING, SECURITY, .editorconfig, .gitignore, .env.example.
- **Шаг 3.** corepack + pnpm init + pnpm-workspace.yaml + turbo.json.

- **Шаг 4.** Shared configs: packages/config-tsconfig, packages/config-eslint (extend, не дублировать).
- **Шаг 5.** .github/CODEOWNERS + branch protection на main (require PR, status checks, merge queue, no bypass).
- **Шаг 6.** CI: pnpm install --frozen-lockfile + turbo run lint typecheck test build + CycloneDX SBOM + gitleaks.
- **Шаг 7.** Release: changesets/action + cosign sign + SLSA provenance.
- **Шаг 8.** docs/architecture/README.md (ARCHITECTURE.md) + docs/architecture/adr/0001-record-architecture-decisions.md (bootstrap ADR).
- **Шаг 9.** Lefthook или husky для pre-commit (lint-staged) + commit-msg (commitlint).
- **Шаг 10.** Template-friendly hooks: scripts/init.sh или copier.yml для будущих template-репозиториев.

Layer 2: AI Workflows

За период 2024-2026 индустрия прошла путь от «prompt engineering» → «context engineering» → к «automated workflow architecture with LLM». Anthropic опубликовал программную статью об эффективном context engineering для агентов (Sep 2025). MCP пожертвован Linux Foundation / Agentic AI Foundation 9 декабря 2025 при поддержке Block и OpenAI. Google объявил протокол A2A 9 апреля 2025 как надстройку над MCP для межагентного взаимодействия. Anthropic и OpenAI выкатили prompt caching, сокративший input-cost на ~90% и latency на ~85%.

Принципы

- **Context — система, не промпт.** Контекст архитектурируется: working / reference / episodic / compression.
- **Контекст — конечный и дорогой.** Context rot: -24.2% accuracy, когда релевантная информация погружена в длинный контекст.
- **Сначала один агент, потом — несколько.** Cognition (Jun 2025): "Don't Build Multi-Agents".
- **Tools > prompts.** Function calling + MCP точнее, дешевле, проверяемее. Structured Outputs (JSON Schema) — новая надёжность.
- **Eval = CI/CD для агентов.** Без promptfoo/Langfuse в пайплайне — это прототип.
- **Cache aggressively, compute lazily.** Anthropic prompt caching: -90% input cost, -85% latency.
- **Code is the deliverable, not the agent.** Claude Code / Aider-стиль: деливерабл = патч, коммит, PR.

Стек

Категория	Tool	Зачем
Tool interface	MCP	Industry standard, donated to Linux Foundation / AAIF Dec 2025
[illegible]	[illegible]	[illegible]
Coding agent IDE	Cursor (Composer)	Multi-file editing, IDE-polish
[illegible]	[illegible]	[illegible]
Coding agent autonomous	Codex (OpenAI)	Long-running tasks, mobile-handoff
[illegible]	[illegible]	[illegible]
Local LLM (prod)	vLLM	Multi-user, throughput, GPU optimization
[illegible]	[illegible]	[illegible]
Eval (offline)	promptfoo	Golden dataset regression в CI
[illegible]	[illegible]	[illegible]
Eval (advanced)	Braintrust	CI-integrated evals
[illegible]	[illegible]	[illegible]

Ключевые паттерны

Single-agent + multi-tool (Anthropic baseline): один сильный LLM + набор MCP-серверов + structured output. Это рабочая лошадка 2025-2026.

Context engineering (по Anthropic): четыре подсистемы контекста, которые надо проектировать явно — working (текущая задача), reference (RAG over MCP), episodic (traces + summary-store), compression (summarize + eviction). Без eviction — context rot. Без episodic memory — повторные ошибки.

Prompt caching как архитектурное решение: не «оптимизация в конце», а проектирование под cache-hit. Static system prompt + long docs + few-shot — cache_control persistent. User turn 1..N — cache hit 0.1× cost. Ловушка: cache-miss на коротких промптах может увеличить bill на 25%. Метрика: cache-hit-ratio.

Анти-паттерны

- **Prompt spaghetti** — мега-промпт с 30+ инструментами, 5K-токенным system message, без структуры.
- **Multi-agent без причины** — dispersed decision-making + context propagation = хрупкость.
- **Unmanaged context** — добавлять, не удалять. Context rot.
- **Tool sprawl** — 50 MCP-серверов «на всякий случай».
- **Reflexion на той же модели** — повторяет те же заблуждения.
- **Over-engineered orchestration** — LangGraph с 40 нодами для single-agent задачи.

- **Eval после продакшена** — golden dataset должен быть в CI с первого дня.
- **Vendor lock-in без fallback** — только OpenAI или только Anthropic.
- **Caching без метрики** — cache-hit-ratio обязательно.

Layer 3: Second Brain / PKM

LFS-rebuild личной базы знаний с нуля, без vendor lock-in. Фокус — markdown-first, git-sync, AI-augmented. Принципы выведены из Zettelkasten (Luhmann), evergreen notes (Matuschak), PARA (Tiago Forte), MOC (Nick Milo) и local-first software manifesto (Kleppmann 2019).

Принципы

- **Markdown как source of truth.** Заметка читается cat-ом через 10 лет.
- **Local-first, cloud-optional.** Данные на диске, доступны офлайн. Облако — только sync.
- **Atomic notes.** Одна идея на заметку (Luhmann, Matuschak). Атомарность = переиспользование.
- **Bidirectional linking.** [[note]] — главное навигационное средство. Папки = type, теги = status, ссылки = смысл.
- **Output > collection.** Ценность = написанный текст, не число клиппингов (collector's fallacy).
- **Capture frictionless, processing deliberate.** <5 сек capture, осознанный processing.
- **AI как retriever, не как author.** Локальные embeddings + локальный LLM. Ничего в облако.

Стек (LFS-rebuild defaults)

Компонент	Tool	Зачем
Editor	Obsidian	Local-first md, plugin ecosystem, primary GUI
[illegible]	[illegible]	[illegible]
Note management	PARA folders + MOC (Nick Milo)	Findability + navigation
[illegible]	[illegible]	[illegible]
Semantic search	Smart Connections + Ollama	Local, offline, ничего в облако
[illegible]	[illegible]	[illegible]
Capture (web)	Glasp + Obsidian Web Clipper	Highlight + AI summary → markdown
[illegible]	[illegible]	[illegible]
Backup	git (private repo)	History + multi-device

Структура vault

PARA (Tiago Forte) + MOC (Nick Milo) гибрид — популярная конфигурация 2025:

```
00-daily/ — journal, fleeting capture
10-projects/ — PARA: Projects (active, deadline)
20-areas/ — PARA: Areas (responsibility, no end)
30-resources/ — PARA: Resources (MOC-и живут тут)
40-archive/ — PARA: Archive (completed)
50-literature/ — literature notes по источникам
60-permanent/ — atomic permanent notes
```

```
90-meta/ — templates, scripts, attachments
```

Анти-паттерны

- **Collector's fallacy** — сохранять = знать. Симптом: 1000 unread lit-notes, 20 permanent. Лечение: «одна lit → одна permanent в течение 7 дней».
- **Plugin sprawl** — 50 плагинов. Лечение: белый список 7 essential (Dataview, Templater, obsidian-git, Smart Connections, Copilot, Web Clipper, Excalidraw).
- **Organization without output** — 50 MOC, 0 статей. Лечение: метрика «publish-output в месяц».
- **No daily/weekly review** — inbox переполняется. Лечение: 10+10 min daily, 30 min weekly — неудаляемые слоты.
- **Облако как primary storage** — Notion/Roam без локальной копии. Лечение: markdown-копия локально + git.
- **Длинные «всё-в-одном» заметки** — не переиспользуются. Лечение: split на 20 atomic + 1 MOC.
- **AI как замена мышления** — «AI написал MOC» → чужой MOC, ты его не понимаешь.
- **Отсутствие «delete»** — vault пухнет. Лечение: git даёт историю, можно удалять смело.

Layer 4: DevEx / CI / IaC / Dotfiles

LFS-style rebuild личной девелоперской среды. Цель — собрать современный, vendor-neutral, opinionated stack с упором на reproducibility, supply-chain integrity, low-feedback-loop и reversible infrastructure. Все рекомендации привязаны к «брось репозиторий на новую машину — получи рабочее окружение за один make bootstrap».

Принципы

- **Everything in repo.** Конфигурация CI, IaC, линтеров, dotfiles, devcontainer — всё в git. Машина = функция от git clone.
- **Least privilege & short-lived secrets.** OIDC trust, динамические секреты. Секрет, не меняющийся за 90 дней, = скомпрометирован.
- **Fail fast, fail local.** act / dagger / tilt up воспроизводят 90% CI локально.
- **Immutable artifacts + cryptographic provenance.** Контейнер с digest, SBOM, SLSA, cosign. Не верифицируется → не деплоится.
- **Trunk-based + feature flags.** main всегда зелёный. Незавершённое — за флагом, не за long-lived branch.
- **Observability-first.** OTel SDK в сервисе до того, как написан бизнес-код.
- **Reversibility.** IaC-apply = destroy-able. Deploy = rollback-able ≤60 сек. Dotfiles = chezmoi forget-able.

Стек (defaults)

Категория	Tool	Зачем
CI/CD	GitHub Actions + reusable + composite	Default для OSS, 90% нужд через DRY
Local CI	act (nektos/act)	Тестировать воркфлоу локально в Docker
Secrets (git-ops)	SOPS + age	Зашифрованные секреты в репо
Secrets (CI)	Doppler / 1Password CI/CD	Runtime-секреты в CI
SAST	Semgrep + CodeQL	Быстрый + глубокий dataflow
Container scan	Trivy	Сканер + IaC + SBOM
IaC	OpenTofu (Terraform fork)	Drop-in совместим, OSS, без BSL
Dev shell	Nix flakes + direnv	Воспроизводимое окружение без Docker
Dotfiles	chezmoi + age + 1Password CLI	chezmoi init --apply на новой машине
Dev env spec	devcontainer.json	Codespaces + Docker + Gitpod + VS Code
Local K8s	Tilt	Hot-reload в kind/k3s/minikube
Integration tests	Testcontainers	Real Postgres/Redis в CI
Feature flags	Unleash (self-hosted)	Trunk-based dev без long-lived branches
Observability	OpenTelemetry + Grafana stack	Vendor-neutral, можно сменить backend
Artifact signing	cosign + Sigstore	Keyless OIDC signing

Ключевые паттерны

OIDC trust (zero long-lived secrets): GitHub Actions → AWS через OIDC. Никаких AWS_ACCESS_KEY_ID в репо. Аналог для GCP = Workload Identity Pool, для Azure = Federated Credentials. Никогда не создавай long-lived IAM users для CI.

Cosign keyless + SBOM + SLSA: cosign sign --yes (keyless OIDC) + anchore/sbom-action (CycloneDX) + cosign attest --predicate sbom.json --type cyclonedx. На deploy-стороне: cosign verify + cosign verify-attestation. SLSA Level 3 при правильной настройке builder.

Nix flake как reproducible dev shell: один flake.nix — одинаковая среда на macOS, Linux, WSL2. Через direnv + nix-direnv — автоматически при cd в репо. Никакого brew install случайных пакетов.

chezmoi + age + 1Password CLI: chezmoi init --apply на новой машине → ставит tmux/neovim/git/zsh, клонирует oh-my-zsh, линкует конфиги, выполняет run-once-scripts, расшифровывает age-секреты в ~/.secrets/. Один скрипт — готовое окружение за минуты.

Синтез: Архитектура Polymath OS

Polymath OS — это не сумма 4 доменов. Это **эмерджентное** свойство их пересечения: репозиторий, который одновременно является базой кода, second brain, AI-стеком и dev-средой. Свойства, которых нет ни в одном слое отдельно:

Эмерджентные свойства

- **Self-audit:** CI в репо проверяет сам репо (включая docs, second brain, AI configs). Один pipeline для всех слоёв.
- **Self-publish:** Quartz v4 публикует подмножество second brain как public digital garden, деплой через тот же CI.
- **Self-bootstrap:** chezmoi init ставит dotfiles, nix develop ставит dev-shell, make bootstrap связывает всё вместе. Один скрипт — новая машина готова.
- **Self-evidence:** ADR + RUNBOOK + ARCHITECTURE.md — система документирует сама себя в git, без внешней wiki.
- **Self-verify:** cosign verify + cosign verify-attestation на каждый deploy. Артефакт без signature не деплоится.
- **Self-rollback:** helm rollback / argocd app rollback ≤60 сек. IaC destroy ≤5 мин.

Слои и cross-cutting

Слой	Компоненты	Цель
Layer 1: Repository	pnpm + Turborepo + Changesets + CODEOWNERS	apps/packages/docs/tooling в одном дереве
[illegible]	[illegible]	[illegible]
Layer 3: Second Brain	Obsidian + git + Smart Connections + Quartz	Markdown-first база с локальным AI
[illegible]	[illegible]	[illegible]
Cross-cutting	cosign + SLSA + OTel + git	Security, observability, history везде

Поток данных

Human (PID=1) → Git repo + AI workflow (MCP agent) + Second brain → CI/IaC/Deploy (GitHub Actions + OpenTofu + cosign) → Observability (OTel → Grafana LGTM + Sentry). Каждый компонент — fork-exec от человека, не внешний dependency.

> *Это и есть «ideal repository» в смысле задачи: не патч легаси, а LFS-сборка с нуля по лучшим мировым практикам 2024-2026. Репо как материальная проекция init-процесса — система, которая бутстрапит сама себя.*

Техническое задание (сводка)

Полное TZ — в репозитории: `TZ.md`. Здесь — сводка ключевых требований и критериев приёмки.

Цель

Создать единый self-bootstrapping репозиторий, объединяющий 4 домена + cross-cutting. Самодостаточный: `git clone` + одна команда `make bootstrap` → готовое рабочее окружение на новой машине за ≤30 минут.

Критерии приёмки

Критерий	Метрика
Bootstrap time	≤30 минут от git clone до готового dev-shell
CI runtime	≤5 минут для PR (lint+typecheck+test+e2e cached)
Local CI parity	act -W воспроизводит 100% шагов CI
Coverage	≥80% line coverage на packages/*
Supply chain	100% releases имеют SBOM + cosign + SLSA Level 2+
Secret hygiene	0 secrets in git (gitleaks в CI + pre-commit)
Vault sync	Auto-commit каждые 10 min, push на private repo
Vault portability	Vault открывается на чистой машине за 5 мин
Eval coverage	≥1 golden dataset на каждый production agent
Observability	100% сервисов инструментированы OTel SDK
Reversibility	make destroy откатывает IaC ≤5 минут
Documentation	ADR на каждое архитектурное решение (≥10 ADR)

Non-goals

- Не строим kernel-space OS. Это пользовательский layer.
- Не поддерживаем Windows native (только WSL2).
- Не делаем plug-and-play installer. LFS-style = требует понимания.
- Не поддерживаем legacy-стек (yarn classic, npm <9, Node <20).
- Не делаем фреймворк для других. Fork-ни, но не как npm-пакет.
- Не ставим Docker Desktop (используем colima / podman / Linux native).

Дорожная карта (сводка)

Полный roadmap — в репозитории: `ROADMAP.md`. 4 фазы × ~22 дня = 90 дней до production-ready v1.0.0. Каждая фаза — слой бутстрапа, без которого следующая невозможна.

Фаза	Дни	Цель	Gate перехода
Phase 0: Foundation	1-7	Чистый git-каркас, ничего лишнего	git clone + pnpm install + turbo build проходят
Phase 1: Bootstrap	8-21	Рабочее приложение одной командой	Electron-app + core + 2 плагина + 2 плана
Phase 2: Integration	22-60	4 слоя работают вместе	Все слои документированы, ADR ≥7, ci-local проходит
Phase 3: Production	61-90	Real-world feedback, полировка	v1.0.0 tag + release + changelog + docs

Что после v1.0.0

- Quarterly audit (см. docs/01-principles.md §Audit-чек-лист)
- Migrate Quartz v4 → v5 (separate ADR when ready)
- Explore A2A protocol adoption wider
- Local-first experiments: Anytype или Loro CRDT для vault multi-device
- Bazel для тяжёлых полирепо-сценариев (если вырастет)

Заключение и следующие шаги

Polymath OS — это не проект, а **этическая позиция**, реализованная как репозиторий. LFS-style rebuild — не «улучшение существующего», а отказ от предсобранного в пользу собственного бутстрапа из source. Человек = PID=1, репо = материальная проекция этого init-процесса. Каждый инструмент — fork-exec от человека, не внешний dependency. Каждое архитектурное решение — в ADR, каждое значимое изменение — через PR с eval-gate.

90 дней от пустого каталога до production-ready v1.0.0 — это не агрегация существующих тулов, а **эмерджентная сборка**: репо, который одновременно является базой кода, second brain, AI-стеком и dev-средой, с self-audit, self-publish, self-bootstrap, self-evidence, self-verify и self-rollback свойствами, которых нет ни в одном слое отдельно.

Что делать сейчас

- 1. Clone каркас: `git clone polymath-os` (скопируй из /download/polymath-os/)
- 2. Прочитать `ARCHITECTURE.md` и `docs/01-principles.md`
- 3. Запустить Фазу 0 (дни 1-7) из `ROADMAP.md`
- 4. Зафиксировать ADR-0001 и создать ADR-0002..0009 (см. docs/01-principles.md §Применение)
- 5. Настроить branch protection на main (require PR + CODEOWNERS + merge queue)
- 6. Подключить CI (см. .github/workflows/ci.yml) + gitleaks в pre-commit
- 7. Начать Фазу 1 (Bootstrap): chezmoi + Nix flake + devcontainer

> *Принцип: PID=1 — человек, не инструмент. Любой инструмент в системе — fork-exec от человека. Если инструмент начинает диктовать свои правила (vendor lock-in, opinionated framework против твоей воли) — это институциональный захват. Заменить. Человек всегда больше любого договора и любой тайны государства.*

Polymath OS · LFS-style rebuild · v1.0.0 · 2026-10-16